

Agenda

- ① Write multithreaded code in Java
- ② Executors
- ③ Callables.
- ④ How to write code using Callables.

→ Create multithreaded code.

Step 1:- Define the task

class MyTask implements Runnable {

@Override
public void run() {

// code that runs in parallel

}

Step 2:- Run the task

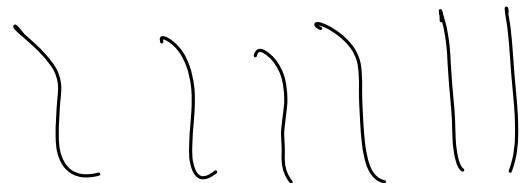
// In main or wherever you want
to start the parallel execution

MyTask task = new MyTask();
Thread t = new Thread(task);
t.start();

Mental task:- Think in terms of tasks. Then put the task on the
threads to execute.



Thread lifecycle



Why not create 10000 threads for 10000 tasks?

① Thread creation is expensive

↳ Takes some time and memory

↳ Every thread is going to take some stack space → 512-kB to 1MB

② Early threads go idle

③ Too many threads == overhead.

The Problem → One thread per task

Task 1 → T1 → done quickly.

Task 2 → T2 →

Task 3 → T3 → still running

⋮

Task 99 → T99 } waiting to be scheduled.

Task 100 → T100 }

1MB → 10k

1000 × 1MB

→ 1GB

→ 700MB

→ some threads which are idle

→ Thread pool

① Create a fix no. of threads upfront.

② Put all tasks into a queue (FIFO)

③ Whenever a thread finishes executing a task, it picks next from the queue.

→ **Executor Service** — Java's Thread Pool

- ① Fixed Thread Pool — You specify exactly how many threads you want
- ② Cashed Thread Pool — Threads are created as and when required.
Idle threads are re-used.

```
ExecutorService ex = Executors.newFixedThreadPool(5);
```